GUIA VISUAL 071

TRATAMENTO DE ERROS SET -EUO PIPEFAIL E TRAPS

*O script de backup rodava havia sete meses e o e-mail do `cron` chegava vazio — sinal de sucesso. No dia da restauração, o arquivo do dia anterior tinha **20 bytes**. Três meses antes o `/mnt/backup` deixara de ser montado: o `mkdir` criou o diretório no disco raiz, o `tar` encheu a partição e morreu, e o script **seguiu obediente** para a linha seguinte — apagou os dumps, imprimiu "backup concluido" e **saiu com status zero**. Nada ali está sintaticamente errado: é o padrão do Bash. **Este assunto é desligar esse padrão.***

071
GUIAS LINUX

OS TRÊS PECADOS DO PADRÃO · O PRÓLOGO QUE OS DESLIGA · E ONDE O SET -E AINDA NÃO AGE

1 ERRO NÃO INTERROMPE
O Bash segue para a próxima linha, aconteça o que acontecer. Um **cd** que falha não impede o `rm -rf *` seguinte de rodar — **no diretório errado**.

2 VARIÁVEL AUSENTE = STRING VAZIA
Erro de digitação não gera erro nenhum: `rm -rf "$DESTNO/"` vira `rm -rf /`. É literalmente a origem do bug que apagou a instalação do Steam em 2015.

3 O PIPELINE MENTE
O status é o do **último** comando. Em `zcat corrompido.gz | wc -l`, se o `zcat` explodir o `wc` imprime `0` e sai com zero: o pipeline inteiro reporta sucesso.

```
#!/usr/bin/env bash
set -Eeuo pipefail
shopt -s inherit_errexit
```

`-e`	**errexit** — morre no primeiro status ≠ 0
`-u`	**nounset** — variável não definida é erro fatal
`pipefail`	status do **último que falhou**, não do último
`-E`	**errtrace** — o trap ERR é herdado por funções e subshells
`inherit_errexit`	fecha o buraco do `x=$(cat /nao/existe; echo ok)`, que **sobrevive** sem ele (Bash ≥ 4.4)

`if / while`	`&& ou ||`	`! comando`	`a | b | c`	SÓ AQUI O SCRIPT MORRE
condição de teste	tudo à esquerda	negação	quem não é o último	fora das quatro, status ≠ 0 encerra na hora

as quatro escapatórias são INTENCIONAIS — sem elas, todo `if` falso mataria o script

A exceção que mais engana é a função. Chamada em contexto de condição, o `set -e` fica desligado **dentro dela inteira, recursivamente**: uma função de 40 linhas invocada como `if preparar; then` roda até o fim mesmo que a terceira linha falhe — imprime `AINDA RODEI` e ainda devolve "tudo certo". Duas regras daí: **não chame função longa em contexto de condição** (separe a checagem do trabalho) e, dentro dela, verifique explicitamente com `cd /alvo || return 1`. A segunda armadilha é aritmética: `((i++))` devolve **status 1 quando o valor anterior era zero** — um contador que começa em `i=0` mata o script. Use `((++i))` ou `i=$((i + 1))`. Sob `set -u`, o idioma de memorizar é `${VAR:-}` para leitura segura e `${1:?uso: $0 <arquivo>}` para argumento obrigatório com mensagem própria. E para saber **qual** estágio do pipeline falhou existe o array `PIPESTATUS` — volátil, sobrescrito até por um `echo`: copie na linha seguinte com `estados=("${PIPESTATUS[@]}")`.

TRAP: A LIMPEZA QUE O SET -E NÃO FAZ

kali@kali: ~/lab-erros — limpeza.sh

```
$ cat limpeza.sh
#!/usr/bin/env bash
set -Eeuo pipefail; shopt -s inherit_errexit
# temporário com credencial: só o dono lê
umask 077
TMP="$(mktemp -d)"          # nunca /tmp/script-$$
limpar() {
    local st=$?              # PRESERVE o status ANTES de tudo
    rm -rf -- "$TMP"         # aspas + -- : se vazio, falha
    return "$st"
}
trap limpar EXIT             # roda SEMPRE (menos em kill -9)
erro() { printf '%s: erro %d na linha %d: %s\n' \
    "${BASH_SOURCE[1]##*/}" "$?" "${BASH_LINENO[0]}" \
    "$BASH_COMMAND" >&2; }
trap erro ERR                # só diagnóstico, nunca limpeza
mountpoint -q /mnt/backup || die 78 "nao montado"
$ ./limpeza.sh --falhar; echo "status: $?"
limpeza.sh: erro 1 na linha 18: false
limpando /tmp/tmp.7Kx2 (status 1)
status: 1
```

Ctrl-C que se comunica. Sair com `exit 1` depois de uma interrupção faz o chamador acreditar em falha genérica. A forma correta de "morrer como se não houvesse trap" é `interromper() { limpar; trap - INT; kill -INT "$$"; }` — o pai enxerga **130** (128+2) e sabe que houve `SIGINT`.

OS QUATRO PSEUDO-SINAIS

`EXIT`	o shell vai terminar **por qualquer motivo** — fim normal, `exit 1`, morte por `set -e`, `SIGTERM`, `SIGINT`. Só `kill -9` escapa. **É onde mora a limpeza**
`ERR`	um comando falhou, sob **as mesmas quatro exceções** do `set -e`. Só diagnóstico
`DEBUG`	antes de **cada** comando simples (herdado com `set -T`)
`RETURN`	ao voltar de uma função ou de um `source`

A ORDEM É FIXA

`ERR` **dispara antes do** `EXIT`, e o `EXIT` é o último código do processo a rodar — mesmo que o handler de `ERR` chame `exit`. O status que o chamador enxerga é **o que o handler de** `EXIT` **devolve**: sem o `local st=$?` na primeira linha, um `rm` bem-sucedido transforma falha em sucesso. Nas aspas do `trap`, **simples e uma função nomeada**: `trap "rm -f $ARQ" EXIT` expande **no registro**, `trap 'limpar' EXIT` expande **no disparo** (é o aviso `SC2064` do ShellCheck).

CÓDIGOS COM SIGNIFICADO

`1–63`	livre para o seu script
`64`	argumento inválido · `65` entrada malformada
`69`	dependência ausente · `78` erro de configuração (`sysexits.h`)
`126/7`	não executável / não encontrado — do shell
`128+N`	morto pelo sinal N: **130** = Ctrl-C, **143** = `SIGTERM`

O QUE O PRÓLOGO **NÃO** FAZ

Não detecta o erro que **não devolve status** (`grep` sem casamento devolve 1 sendo normal; `find` sem resultado devolve 0). Não valida entrada. Não garante atomicidade — só o `trap`, ou escrever num temporário e `mv` no fim. E não substitui a verificação explícita: `mountpoint -q /mnt/backup || die 78 …` é o que teria salvado o script da abertura. **É o piso, não o teto:** converte falha silenciosa em falha ruidosa, e é só isso que se pede dele.

O caso `rm -rf` guiado por variável é a pior consequência dos pecados 1 e 2 juntos. Sem `set -u`, `rm -rf "$DESTNO/"` com o nome digitado errado apaga a raiz; sem aspas e sem `--`, um `$TMP` vazio ou começando com `-` vira comando de destruição ou erro de sintaxe silencioso. A forma segura é **sempre** `rm -rf -- "$TMP"`, com `TMP` vindo de `mktemp -d` — nome previsível em `/tmp` é vulnerabilidade clássica de link simbólico: o atacante cria antes `/tmp/meuscript-1234` apontando para `/etc/shadow` e o seu script escreve lá com o privilégio que tiver. **Ensaie qualquer trap de limpeza com `echo rm -rf --` no lugar do `rm`** antes de rodar de verdade, e ensaie em VM descartável.

RESUMO RÁPIDO

- ✓ **O padrão do Bash é continuar depois do erro**, tratar variável ausente como vazia e reportar só o último do pipeline. Os três juntos produzem o script que falha e diz que deu certo.
- ✓ **`set -Eeuo pipefail` + `shopt -s inherit_errexit`** é o prólogo completo: as três correções mais a herança do trap `ERR` e o subshell de `$( )`.
- ✓ **`set -e` tem quatro exceções intencionais** — condição de `if` / `while`, esquerda de `&&` / `||`, negação com `!` e comando não final de pipeline. A pior é a **função chamada em contexto de condição**, que roda inteira sem proteção.
- ✓ **`((i++))` com `i=0` devolve 1** e mata o script. Use `((++i))` ou `i=$((i + 1))`.
- ✓ **`${VAR:-}` é o acesso seguro sob `set -u`**; `${1:?mensagem}` documenta o argumento obrigatório na própria expansão.
- ✓ **A limpeza mora no `trap … EXIT`**, sempre com `local st=$?` na primeira linha do handler e `mktemp -d` em vez de nome previsível. O `ERR` cuida só do diagnóstico, e dispara antes.
- ✓ **`trap - INT; kill -INT "$$"`** é a forma correta de morrer por interrupção: devolve 130 em vez de um `1` mudo.
- ✓ **Nenhuma opção do `set` substitui verificação explícita** — `mountpoint -q`, `command -v` e `die` com código significativo continuam sendo trabalho seu.

NO KALI

Ferramenta cujo status 1 significa "não achei" é a briga diária com o `set -e`. `nmap` com host fora do ar, `grep` sem casamento, `nikto` sem achado — todos devolvem status ≠ 0 em situação **normal** de um teste, e o script morre no primeiro alvo silencioso. O idioma é isolar e decidir pelo conteúdo, não pelo status: `resultado=$(nmap -Pn "$alvo") || true`. **Script de coleta que morre no meio deixa evidência pela metade:** o `trap EXIT` não deve só apagar temporários — tem de fechar o arquivo de resultado de forma consistente e **registrar no log que a execução foi interrompida**, porque um relatório truncado sem marca de interrupção é pior que nenhum: parece completo. Se o script manipula hash, token ou arquivo de senha, o `mktemp -d` vem precedido de `umask 077` e a limpeza usa `shred -u` onde o sistema de arquivos permitir. **Num script que toca alvo externo, o `trap INT` tem de parar de verdade** — e parar também os filhos: continuar gerando tráfego depois que o operador mandou parar é, na prática, exceder o escopo combinado (Lei 12.737/2012). E **verificação de escopo é `die`, não aviso**: a checagem de que o alvo está na lista autorizada encerra o script com código distinto, nunca imprime alerta e prossegue — aviso em `stderr` dentro do `cron` não é lido por ninguém. **Família RHEL:** `set -e` e `trap` são do Bash, mas `/bin/sh` ali é o Bash em modo POSIX, sem `inherit_errexit` — shebang `#!/bin/sh` perde as proteções nas duas famílias, e `#!/usr/bin/env bash` é a forma portátil. Em serviço do systemd, o status de saída é o que dispara `Restart=on-failure`: sair com zero depois de falhar impede o serviço de reiniciar.